



UNIVERSITY OF
LEICESTER

MOBILE AND WEB APPLICATIONS

GARIGIPATI YASWANTH SAI

ysg2@student.le.ac.uk

259044153

1.a.

When merging XML documents from various source vocabularies, prefixes and namespaces help to uniquely identify elements and attributes and avoid naming conflicts.

Namespaces : A group of names denoted by a URI (Uniform Resource Identifier) is called a namespace. It serves as a distinct identifier to distinguish between elements that may have the same local name but distinct semantics (e.g., a <title> element in an HTML namespace vs. a <title> element in a book metadata namespace).

Prefix: A shorthand for a namespace URI that is used inside an XML document to improve the readability and cleanliness of the markup. For instance, d:element denotes that the XML Schema Definition namespace has the element tag.

1.b.

The document is **not well-formed** due to the following errors:

- The attribute name=Huntr/x is not enclosed in quotation marks.
- The schedule element contains duplicate attributes with the same name day.
- Element names such as <20260110> start with digits, which is invalid in XML.
- The ampersand character (&) in Rumi, Mira & Zoey is not escaped as &.
- There are elements and text appearing outside the single root element.
- The <note> tag is incorrectly nested and not properly closed.

1.c.i.

The statement searches all <folder> elements in the document and checks whether their combined text content contains the string "xsd".

The contains(.) function evaluates the text of the folder and all its child elements.

The final step retrieves the name attribute of any matching folder.

In this document, the folder **Coursework1** contains the file CW1.xsd, so its name is selected.

Matching folder: name="Coursework1" on **line 5**.

(2) count(//folder[@name="Coursework1"]/file)

- This XPath locates the folder named Coursework1 and counts how many file elements it contains directly.
- Only immediate child <file> elements are included in the count.
- Since the folder contains three files, the output is these files CW1.xsd, CW1.xslt, and CW1.py.

1.c.ii.

1. Select all file names (excluding folders) - //file
2. Select any files ending with “.py” extension- //file[ends-with(text(), ".py")]
3. Select all non-empty folders - //folder[file or folder]
4. Calculate the average number of files directly contained in each folder - avg(//folder/count(file))

1.c.iii.

```
{
  "file": "note.xml",
  "folder": {
    "@name": "Coursework1",
    "file": ["CW1.xsd", "CW1.xslt", "CW1.py"]
  }
}
```

1.c.iv.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="folder">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="file" type="xs:string" minOccurs="0" maxOccurs="unbounded"/>
        <xs:element name="folder" minOccurs="0" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="file" type="xs:string" minOccurs="0"
maxOccurs="unbounded"/>
            </xs:sequence>
            <xs:attribute name="name" type="xs:string" use="required"/>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
      <xs:attribute name="name" type="xs:string" use="required"/>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

1.d.

- i. **False** : Uninitialised variables are undefined, not "null".
- ii. **True** : REST can use JSON, XML, etc. SOAP typically uses XML envelopes/messages.
- iii. **False** : Node.js is mainly a **server-side** runtime (can be used for tooling too), not client-side.
- iv. **False** : Android SDK is for Android. iOS requires separate toolchains (unless using cross-platform frameworks).
- v. **False** : DOM loads the whole document into memory, so SAX is generally more memory efficient.
- vi. **False** : SPAs can be indexed if rendered appropriately (server-side rendering, pre-rendering, dynamic rendering).
- vii. **True** : REST calls are designed to be stateless: each request contains what the server needs.
- viii. **True** : If a user blocks cookies or rejects consent, cookies may not be stored, which can prevent cookie-based sessions from working.

1.e.

1.

- The expression `1 + "1"` forces numeric 1 to become a string, resulting in `"11"`.
- A block is then created.
- The `var h` inside the block is not block-scoped, so it changes the global `h` value to 0.
- The `let i` declaration is block-scoped, so it does not affect the outer `i`.
- After the block ends, `h` stays 0 and `i` is still true.

1.e. ii.

In a loosely typed language, variables can hold values of different types and implicit type conversion occurs. The expression `var h = 1 + "1"` demonstrates this, as JavaScript converts the number into a string and performs concatenation.

1.e.iii.

Arrow functions are anonymous functions introduced in ES6 that simplify function expressions and inherit this from their surrounding scope.

```
function check(a, b) {  
  return typeof a === typeof b;  
}
```

1.e. iv

- No error occurs, the output is - `output=true0NaN3trueundefined`

1.f - JQuery Selectors

- i. `$("li a")`
- ii. `$("div.c2")`
- iii. `$("#c3")`

Question 2.

2.a.

Progressive enhancement is a design methodology in which a website is constructed beginning with a simple, dependable experience that functions on all platforms, particularly mobile phones with constrained capabilities. HTML is used first to supply essential content and functionality, guaranteeing efficiency and accessibility. For devices and browsers that allow them, additional capabilities like sophisticated layouts, animations, and interaction are subsequently overlaid on using CSS and JavaScript.

2.b.

- i. **eBusinessMeeting app – Hybrid / Cross-platform**^[SEP] Using a shared codebase reduces development time and cost while still allowing deployment across multiple platforms. Security features such as biometric authentication can be implemented using platform-specific plugins.
- ii. **Self-driving Car Share app – Native**^[SEP] Native apps provide full access to hardware features including GPS, Bluetooth, and secure payment systems. They also offer better performance and reliability, which is essential for safety-critical transport applications.
- iii. **Emergency Prescriptions app – PWA**^[SEP] A PWA allows patients to access the service immediately without downloading an app.
- iv. **Scapa Flow Guide app – Native**^[SEP] The need for offline maps, audio guides, and AR-based reconstructions, makes Native app a good choice for this app

2.c.

MVM Architecture

To increase maintainability and testability, the MVVM (Model–View–ViewModel) architecture divides an application into three different layers. Data management, business rules, and database or external service interactions fall under the purview of the model. The View has very little logic and just concentrates on the display and user interface. By providing data in a format appropriate for display and managing user interactions, the ViewModel serves as a link between the Model and the View. Without directly relying on the Model, the View can immediately update when data in the ViewModel changes thanks to

data binding. This division lessens tight coupling, streamlines UI code, and facilitates testing and development of the application.

2.d.

- i. **True**
- ii. **True**
- iii. **False**
- iv. **True**
- v. **False**

2.e.i.

```
@RequestMapping(value="/NHS_Service/GP/{postcode}", method=RequestMethod.GET)
@ResponseBody
public List<Surgery> getSurgeryByPostcode(@PathVariable("postcode") String pc) {
    List<Surgery> result = new ArrayList<>();
    for (Surgery s : nhs_service.getAllSurgeries()) {
        if (s.getPostcode().startsWith(pc)) {
            result.add(s);
        }
    }
    return result;
}
```

2.e.ii.

```
$("#postcode").on("keyup", function(){
    let pc = $(this).val();
    $.get("/NHS_Service/GP/" + pc, function(data){
        let html = "";
        data.forEach(s => {
            html += "<p>" + s.surgery + " – " + s.currentWaitingTimeMins + " mins</p>";
        });
        $("#surgeries").html(html);
    });
});
```

2.e.iii.

The webpage communicates asynchronously with the REST API via JavaScript and Ajax. An event listener is triggered when the user inputs a postcode prefix, and it uses the given URL pattern to perform a GET request to the REST API. After receiving this request, the server-side controller uses the postcode prefix to filter the list of GP surgeries and sends the results in JSON format. The Ajax success callback on the client side interprets the JSON response and dynamically modifies the HTML content to show the names of the surgeries and the

waiting times as of right now. This method offers a responsive user experience and prevents page reloads.

2.f.

Identity proofing and onboarding (high assurance):

Users verify identity via document scanning (such as passport, BRP, driving licence) and liveness checks. We implement with camera access, OCR/MRZ scanning, NFC chip reading (ePassport), and a certified identity verification service. Save only necessary verified attributes.

Biometric authentication for access: Use Face ID / Touch ID (iOS) and BiometricPrompt (Android) with keys stored in Secure Enclave/Android Keystore. Use biometrics to unlock the app and for sensitive actions.

Verifiable credentials and selective disclosure: Issue signed digital credentials (like age, name, residency) using W3C Verifiable Credentials. Use cryptographic signatures and let users share only what is necessary (selective disclosure / zero-knowledge proof style if possible).

Consent-driven sharing with third parties: Provide explicit consent screens and short-lived access tokens. Cryptographic proofs are verified by third parties without storing personal data, ensuring GDPR compliance. Provide audit logs and revoke controls.

Online and offline verification: Online: API-based verification with nonce and signature validation. Offline: QR code containing a signed, time-limited proof; verifier app checks signature using published government public keys.

Integration with government portals: Use OAuth 2.0 / OpenID Connect for login into NHS, council, and other services. Support strong assurance flows and step-up authentication when needed.

Security, privacy, and compliance: Data should be encrypted at rest, use TLS, certificate pinning, and device attestation. Include fraud detection signals, lockout policies, and secure backups. Offer clear data minimisation, transparency, and deletion controls.